

Architecture

TP3 Discussion and Staff-Support System

Draft architecture outline based on the current TP3 codebase and the TP3 deliverable requirements

Purpose	Select the architectural views that best communicate the long-term goals, objectives, and intents of the TP3 system, and explain how those views support both the received and derived requirements.
Architecture Focus	The current codebase suggests a layered academic discussion system that now includes staff-facing administration, report review, role control, participation analytics, and controlled access to account operations.

1. Selected Architectural Views

The following architectural views were selected because they best communicate the system's long-term intent: how users and staff interact with the system, how the code is partitioned into reusable layers and services, how data flows through the application, and how quality and authorization concerns are preserved as the project grows.

View	Why This View Was Selected	Primary Requirements Covered	Main Quality / Reuse Goals
System Context View	Shows the major actors and external interactions around the application.	First-admin setup, login, invitation-based onboarding, role dispatch, staff administration, discussion review, report handling.	Scope clarity, stakeholder alignment, outward interfaces.
Logical Layered / Component View	Shows how the system is partitioned into presentation, application/service, domain, analytics, and persistence responsibilities.	All major received and derived user stories.	Maintainability, reuse, separation of concerns, easier refactoring.
Interaction / Data Flow View	Explains how key operations move through controllers, services, entities, and the database.	New account creation, role updates, discussion actions, report review, staff analysis.	Traceability, testability, controlled data movement.
Runtime / Deployment View	Explains the actual executable environment used by this project.	Startup, single-user runtime behavior, H2 persistence, JavaFX client behavior.	Operational realism, persistence assumptions, release readiness.

2. View A - System Context View

Figure A1. UML-style context view (textual).

The system sits at the center of five recurring stakeholder interactions: administrators, staff/instructional team members, role-based end users, the local H2 persistence layer, and internal analytic/reporting support.

Actor / External Element	Main Interaction with the TP3 System	Architectural Importance
First system administrator	Creates the initial admin account when the system is empty.	Avoids hard-coded credentials and establishes secure ownership at bootstrap.
Existing user with invitation code	Uses invitation-based account setup to create a role-constrained account.	Supports controlled onboarding and role assignment.
Administrator / staff reviewer	Views users, changes status/roles, resets passwords, reviews reports, and opens discussion workflows.	Covers the new staff-facing epics and operational support needs.
Role1 / Role2 user	Views role home page, updates account data, and participates in discussions.	Preserves the original role-based structure while allowing later staff additions.
Discussion/report subsystem	Stores posts, replies, report counts, and content lookup by ID.	Provides moderation, traceability, and review support.
Analytics / participation subsystem	Computes threshold and coverage summaries for staff assessment workflows.	Supports derived TP3 needs around fair discussion assessment.
H2 database	Persists users, invitations, posts, replies, and reports.	Provides the durable state required by the application.

Architecture interpretation.

- 2.1. The context view shows that TP3 is no longer only a student discussion tool; it is becoming a staff-support and review system built on top of the TP2 discussion foundation.
- 2.2. The first-admin and invitation mechanisms control who may join and under what role, which directly supports the system's access-control goals.
- 2.3. The staff side of the context now includes user management, discussion review, report inspection, and assessment-oriented analysis rather than only student posting/replying.

3. View B - Logical Layered / Component View

Figure A2. Layered architecture view (packages and responsibilities).

Layer	Representative Packages / Classes	Responsibilities
Presentation Layer	guiFirstAdmin, guiUserLogin, guiNewAccount, guiMultipleRoleDispatch, guiAdminHome, guiRole1, guiRole2, guiDiscussionBoard, guiUserUpdate, customGuiClasses	Constructs JavaFX views, gathers user input, presents tables/dialogs, and routes users into role-specific flows.
Application / Service Layer	Controller* classes, AdminService, AdminServiceImpl, dispatch helpers	Validates actions, coordinates UI requests, applies authorization checks, and orchestrates persistence operations.
Domain Layer	User, Post, Reply, CreateInfo, DiscussionBoardContent, ReportInfo, Role, UserSummary	Defines the system's core data model and reusable business entities.
Analytics Layer	StudentCoverageAnalyzer, StudentCoverageResult, StudentCoverageSummary (prototype direction)	Summarizes student participation and supports staff assessment decisions.
Persistence Layer	Database, H2 schema/tables	Stores and retrieves users, invitations, posts, replies, reports, and supporting state.

Why this layered structure was selected.

- 3.1. The presentation layer remains close to the FoundationsS26 pattern, which maximizes reuse of the provided architecture and lowers integration risk.
- 3.2. Controllers and services provide a clearer place for authorization and staff workflow decisions than embedding those rules inside view code.
- 3.3. Shared domain objects such as Post, Reply, and DiscussionBoardContent reduce duplication and allow report/review logic to work across content types.
- 3.4. The analytics layer isolates staff-assessment logic so that participation calculations can evolve without destabilizing the basic discussion-board flow.

4. View C - Interaction and Data-Flow View

Figure A3. Representative end-to-end flows.

Flow	Entry Point	Core Path	Architectural Significance
Bootstrap admin creation	ViewFirstAdmin / ControllerFirstAdmin	GUI input -> controller validation -> User entity creation -> Database.register -> role routing	Supports secure initialization and removes dependence on hard-coded default credentials.
Invitation-based account setup	ViewNewAccount / ControllerNewAccount	Invitation code lookup -> field validation -> User creation with invited role -> persistence -> account update / home page dispatch	Separates onboarding from runtime role use and supports controlled growth of the user population.
Discussion authoring and moderation	ViewDiscussionBoard / ControllerDiscussionBoard	GUI dialogs -> content validation/spam/language checks -> Post/Reply entities -> Database persistence -> refreshed table views	Shows the main student-side content path and the reason the discussion board remains the central reusable subsystem.
Report review	Admin home / discussion report views	Admin action -> ReportInfo lookup -> Database.getContentByld / report retrieval -> staff inspection	Supports staff visibility into problematic content and creates traceable moderation workflows.
User administration	AdminUsersDialog / AdminServiceImpl	GUI selection -> service authorization logic -> Database authorized update methods -> refreshed user table	Moves sensitive account operations toward a service-based architecture rather than direct ad hoc GUI updates.
Participation analytics	Analytics prototype / staff workflow	Posts + replies -> analyzer -> coverage summaries / thresholds -> staff interpretation	Covers derived TP3 needs around fair and helpful assessment.

5. View D - Runtime / Deployment View

Figure A4. Runtime deployment view (current project reality).

(Single Java/JavaFX desktop client)-> application packages/controllers/services -> local
|H2 database file
()-> generated Javadocs / tests / manual procedures

- 5.1. The current deployment is a single-user desktop-style runtime using JavaFX for presentation and H2 for persistence.
- 5.2. The architecture should therefore be documented as a local rich-client application rather than a web or distributed multi-tenant system.
- 5.3. This deployment choice favors simplicity, instructional reuse, and fast iteration, but it also means concurrency, multi-user session isolation, and enterprise-grade deployment concerns are outside the current scope.

6. Multilevel Outline - How the Architecture Covers the Requirements

6.1 Received functional goals

4. 6.1.1 The architecture supports account creation, login, and role-based navigation through distinct startup, onboarding, and dispatch views.
5. 6.1.2 The architecture preserves the TP2 discussion capabilities: creating posts, creating replies, editing owned content, deleting owned content, reopening/resolving posts, and viewing threaded replies.
6. 6.1.3 The architecture supports admin control of invitations, user review, and role-aware account workflows.

6.2 Derived functional goals for TP3 staff support

7. 6.2.1 The architecture adds staff-facing review paths so that administrators or instructional staff can inspect reported discussion content.
8. 6.2.2 The architecture supports staff management of user activation/deactivation, password reset, and role changes through a dedicated admin dialog and service layer.
9. 6.2.3 The architecture provides a place for participation-analysis logic so staff can determine whether students have replied to enough distinct peers.
10. 6.2.4 The architecture separates review/analysis concerns from ordinary student posting concerns so that the system can evolve without collapsing all responsibilities into one GUI package.

6.3 Non-functional (quality) goals

11. 6.3.1 Maintainability: a layered structure keeps UI changes, service logic, domain logic, and persistence logic from being tightly coupled.
12. 6.3.2 Testability: controller/service boundaries and domain objects give natural seams for JUnit, authorization tests, and analytics tests.
13. 6.3.3 Traceability: entity IDs, report objects, and database lookup methods support connecting visible staff operations to stored records.
14. 6.3.4 Security / authorization: the architecture creates explicit places for role and ownership checks, even if some implementation details still need refinement.
15. 6.3.5 Reuse: the design intentionally extends the provided FoundationsS26 model instead of replacing it with an entirely different architecture.

6.4 Reuse goals and objectives

16. 6.4.1 Reuse of the provided Foundations startup, page structure, and MVC conventions lowers integration risk and keeps the code understandable to graders.
17. 6.4.2 Reuse of generic domain concepts such as DiscussionBoardContent and ReportInfo reduces duplication across post, reply, report, and review workflows.
18. 6.4.3 Reuse of validator packages and custom dialogs reduces repeated validation and GUI boilerplate across onboarding and update flows.
19. 6.4.4 Reuse of service abstractions, especially on the admin side, makes it easier to attach tests to sensitive account-management logic.

7. Architectural Trade-offs and Rationale

- 7.1 The current architecture favors a simple desktop deployment and explicit JavaFX pages over a more abstract but harder-to-finish framework. This is appropriate for the course project timeline and the existing Foundations code.
- 7.2 The project currently mixes view-driven controller logic with newer service-based logic. That trade-off allowed the team to extend TP2 quickly, but long-term maintainability is better when sensitive admin operations continue moving into service methods.
- 7.3 The architecture keeps analytics as a distinct concern. This avoids polluting the discussion-board UI classes with assessment calculations and supports future extension of staff views.
- 7.4 The architecture uses a central Database class and a local H2 file-backed store. This simplifies the project and supports persistence, but it also means the documentation must describe the system as single-user/local rather than enterprise or cloud-native.

8. Architectural Risks Still to Watch

- 8.1 Authorization checks must remain consistent across GUI controllers, service classes, and database update methods.
- 8.2 Role naming and role semantics should remain aligned across UI labels, enum values, persistence methods, and user stories.
- 8.3 New staff features should continue to attach to reusable domain/service seams rather than becoming embedded only in UI code.
- 8.4 The architecture documentation should be kept aligned with any refactoring performed during TP3 so the diagrams and the code continue to flow logically.